

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías



División de Tecnologías para la Integración CiberHumana

Ingeniería en Computación

Programación de Sistemas Avanzados

D01 – IL377 – 223599 – 2026-A

Actividad 2: Primer Avance

Profesor: López Arce Delgado Jorge Ernesto

Integrantes del Equipo:

- Agosto Aceves Carolina
- Garcia Herrera Juan Pablo
- Juárez Rubio Alan Yahir
- Renteria Elias Elisa Yanneth
- Valenzuela López Jorge Yahir

14 de febrero de 2026

Índice

Objetivo.	3
Introducción.	3
Diagrama de la Topología de Red.	4
Roles de los Nodos.	5
Site WireGuard.	5
Site 1 (10.5.5.2/32)	5
Site 2 (10.5.5.3/32)	5
Site 3 (10.5.5.4/32)	6
Site 4 (10.5.5.5/32)	6
Site 5 (10.5.5.6/32)	7
Llaves Generadas de WireGuard.	8
Conexión a la VPN.	11
Instalación de Herramientas Esenciales.	13
Dockerfile, Docker-compose y Kubernetes.	15
Dockerfile	15
Docker-compose.yml	15
Kubernetes	17
Orquestación de kubernetes	18
Pruebas de conectividad de iperf3.	21
PRUEBAS DE KUBERNETES.	24
Problema: El Conjunto de Mandelbrot.	25
Usos y aplicaciones	25
algoritmo en rust.	26
ejecución del programa.	41
Evidencia de la Metodología Scrum.	44
Principales retos técnicos y aprendizajes.	46
Problemas Encontrados y Decisiones Técnicas Tomadas.	47
Configuración de la VPN en Wireguard del lado del Servidor	47
Repositorio en Github.	47
Conclusión.	48
Referencias.	48

Objetivo.

El objetivo de esa actividad no es solo generar el fractal, sino poder demostrar la computación distribuida en un clúster conectado por VPN, usando contenedores y orquestación. En otras palabras, probar que varias computadoras pueden cooperar para resolver un problema pesado. el objetivo de esta actividad es la implementación de un sistema computación distribuida que nos permita generar en este caso una imagen fractal de mandelbrot implementando múltiples computadoras conectadas a través de una vpn creada por wire guard, utilización de contenedores por medio de docker, y orquestación de kubernetes para la distribución de tareas entre los diferentes sistemas de cómputo.

Introducción.

El presente documento describe la implementación completa de un sistema distribuido para el cálculo del conjunto de Mandelbrot, utilizando una arquitectura basada en VPN con WireGuard, contenedores Docker y orquestación mediante Kubernetes (k3s). El sistema consta de un servidor VPN central (hub) y 5 nodos cliente (sites), cada uno ejecutando 4 workers en contenedores, totalizando 20 unidades de cómputo distribuidas. La comunicación segura entre nodos se logra a través de una topología hub-and-spoke implementada con WireGuard, mientras que la orquestación de los workers se realiza con k3s. El algoritmo implementado en Rust divide el plano complejo en franjas horizontales, distribuye el cálculo entre los workers y reconstruye la imagen final del fractal. Las pruebas de rendimiento con iperf3 demuestran un ancho de banda estable de ~95 Mbits/s entre nodos, y las pruebas de ping muestran latencias promedio de 21-104 ms, validando la viabilidad de la arquitectura para cómputo distribuido.

Diagrama de la Topología de Red.

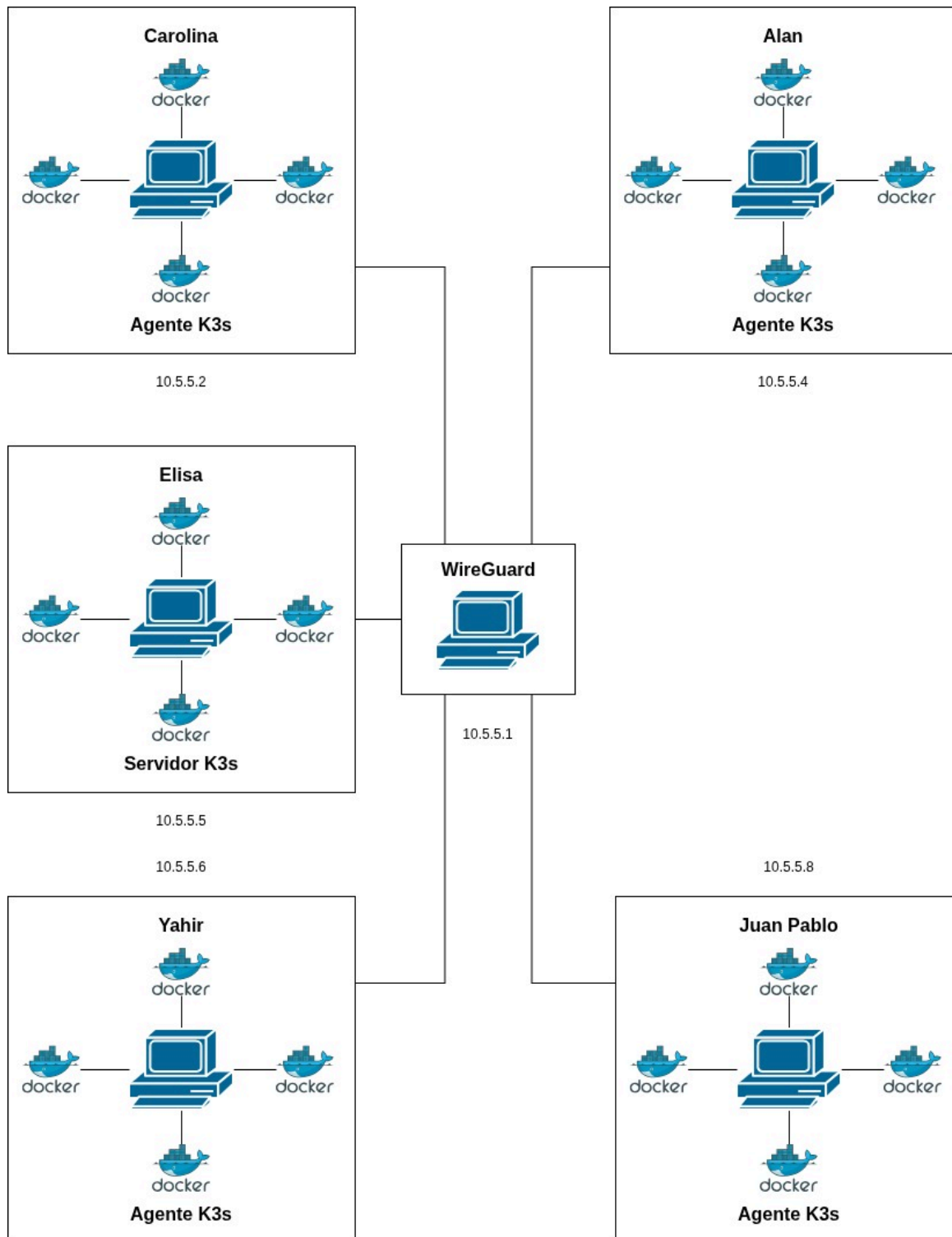


Fig. 1. Topología de la Red Hub-and-spoke

Roles de los Nodos.

Site WireGuard.

Rol: Hub central de la VPN (10.5.5.1/32)

Función: ser el servidor Wireguard el cual se encargará de recibir las conexiones de:

- Site 1
- Site 2
- Site 3
- Site 4
- Site 5

Site 1 (10.5.5.2/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 1
- Nodo 2
- Nodo 3
- Nodo 4

Site 2 (10.5.5.3/32)

Rol del host.

- peer de WireGuard

- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 5
- Nodo 6
- Nodo 7
- Nodo 8

Site 3 (10.5.5.4/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 9
- Nodo 10
- Nodo 11
- Nodo 12

Site 4 (10.5.5.5/32)

Rol del host.

- peer de WireGuard

- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 13
- Nodo 14
- Nodo 15
- Nodo 16

Site 5 (10.5.5.6/32)

Rol del host.

- peer de WireGuard
- Cliente de la VPN
- Host Docker
- Ejecutar 4 workers

Rol de los nodos del Docker:

- Workers de computo
- ejecutar tareas distribuidas
- se comunican únicamente por la VPN

Nodos:

- Nodo 17
- Nodo 18
- Nodo 19
- Nodo 20

Llaves Generadas de WireGuard.

```
elisa@MSI:/mnt/c/Users/elisa$ sudo ls -la /etc/wireguard/
[sudo] password for elisa:
total 36
drwx----- 2 root root 4096 Feb 11 19:05 .
drwxr-xr-x 89 root root 4096 Feb 15 14:33 ..
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_alanyahir.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_caro.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_elisaelias.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_jorgeyahir.conf
-rw-r--r-- 1 root root 251 Feb 11 17:28 client_juanpablo.conf
-rw----- 1 root root 45 Jan 30 12:32 private.key
-rw----- 1 root root 918 Feb 11 19:05 wg0.conf
```

Fig. 2. Directorio de claves generadas para la VPN

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/private.key
ONYfQsTdEbUUDiUsj20seA8BruKZGXF4sHhyKwYcfW4=
```

Fig. 3. Clave privada del servidor

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_caro.conf
[Interface]
PrivateKey = cJSLBVldFnzoKHiZOVv5ITueVXg2QeOCWg72/IcQGVI=
Address = 10.5.5.2/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 4. Claves para Carolina

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_juanpablo.conf
[Interface]
PrivateKey = CBt767PzzL0SIfI0JfMhvNeZIqwcya3m+4Gr5RmViFg=
Address = 10.5.5.3/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 5. Claves para Juan Pablo


```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_alanyahir.conf
[Interface]
PrivateKey = yJ0vDFR89fgHnjF7oJruySuTmQ+k0yB63gQ5ALCe4HE=
Address = 10.5.5.4/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 6. Claves para Alan Yahir

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_jorgeyahir.conf
[Interface]
PrivateKey = QHMFFJEG7A0T6/bLHx4pb/LBARvUImrC19fUmTH9nFI=
Address = 10.5.5.6/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 7. Claves para Jorge Yahir

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/client_elisaelias.conf
[Interface]
PrivateKey = OMFwtgtDCt7z4m8P1RYgky8udm7SDmhh17Si0F63+HM=
Address = 10.5.5.5/32
DNS = 8.8.8.8

[Peer]
PublicKey = 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
Endpoint = 187.139.127.50:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Fig. 8. Claves para Elisa Elias

```
elisa@MSI:/mnt/c/Users/elisa$ sudo cat /etc/wireguard/wg0.conf
[Interface]
Address = 10.5.5.1/24
ListenPort = 51820
PrivateKey = ONYfQsTdEbUUDiUsj20seA8BruKZGXF4sHhyKwYcfW4=

PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o eth4 -j MASQUERADE

PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o eth4 -j MASQUERADE

[Peer]
PublicKey = xPTPf4NfKDJMSZ8lpEFutAxCN/kGgnORLvr4WldFBwc=
AllowedIPs = 10.5.5.2/32

[Peer]
PublicKey = mHVnV0QDZW6P3UkhKxpW4mKu6A1RK9eoK9QAvXT2Rjo=
AllowedIPs = 10.5.5.3/32

[Peer]
PublicKey = C4rPIuZhUh+4XvXgxasngX3kqqlPCoLVHxGYbBCpBA0=
AllowedIPs = 10.5.5.4/32

[Peer]
PublicKey = E1127VnkucLMvpvh165CUdRhc3Rkk09xYkwQ/XP08X0=
AllowedIPs = 10.5.5.5/32

[Peer]
PublicKey = Zg7+rCi4vgToBzc0uHuK7CkxnkjFNwki0dDxkaU06kM=
AllowedIPs = 10.5.5.6/32
```

Fig. 9. Archivo de Configuración para la VPN

Conexión a la VPN.

Para que todos los nodos puedan conectarse al Servidor VPN en Wireguard se realizó la instalación de Wireguard en cada máquina virtual de los distintos nodos. Utilizando el comando `sudo apt install wireguard` para realizar la instalación.

Se debe crear una carpeta para Wireguard con el comando `sudo mkdir -p /etc/wireguard` y crear el archivo de configuración con el comando `sudo nano /etc/wireguard/wg0.conf`, después se debe ajustar los permisos con `sudo chmod 600 /etc/wireguard/wg0.conf`. Para levantar la VPN se utiliza la instrucción `sudo wg-quick up wg0` y verificar la conexión con `wg-show`.

```
carol@DESKTOP-7V7JTD4:~$ sudo wg show
interface: wg0
  public key: xPTPf4NfKDJMSZ8lpEFutAxCN/kGgnORLvr4WldFBwc=
  private key: (hidden)
  listening port: 58473
  fwmark: 0xca6c

peer: 36ANUqXr4lbvke+IaSGVDKmpBmKiPNfWPbjP/PFpvgE=
  endpoint: 187.139.127.50:51820
  allowed ips: 0.0.0.0/0
  latest handshake: 16 seconds ago
  transfer: 92 B received, 180 B sent
  persistent keepalive: every 25 seconds
carol@DESKTOP-7V7JTD4:~$
```

Fig. 10. Conexión de un nodo establecida con el servidor

```
PS C:\WINDOWS\system32> & "C:\Program Files\WireGuard\wg.exe" show
>>
interface: wg0_server
  public key: 36ANUqXr4lbvke+IaSGVDKmPBmKiPNfWPbjP/PFpvgE=
  private key: (hidden)
  listening port: 51820

peer: xPTPf4NfKDJMSZ8lpEFutAxCN/kGgnORLvr4WldFBwc=
  endpoint: 187.139.254.128:55286
  allowed ips: 10.5.5.2/32
  latest handshake: 4 minutes, 37 seconds ago
  transfer: 2.22 KiB received, 1.33 KiB sent

peer: Zg7+rCi4vgToBzcOuHuK7CkxnkjFNwki0dDxkaU06kM=
  allowed ips: 10.5.5.6/32

peer: mHVnV0QDZW6P3UkhKxpW4mKu6A1RK9eoK9QAvXT2Rjo=
  allowed ips: 10.5.5.3/32

peer: E1127VnkucLMvpvh165CUdRhc3RKk09xYkwQ/XP08X0=
  allowed ips: 10.5.5.5/32

peer: C4rPIuZhUh+4XvXgXasngX3kpqlPCoLVHxGYbBCpBA0=
  allowed ips: 10.5.5.4/32
```

Fig. 11. Conexión del lado del Servidor VPN con Wireguard

Instalación de Herramientas Esenciales.

La configuración del entorno es esencial para poder establecer el Sistema Distribuido por lo que es necesario de diversas herramientas que permitan garantizar la conectividad, el despliegue y la evaluación del rendimiento. A continuación, se muestran cada una de las herramientas que fueron instaladas específicamente para el desarrollo del proyecto.

Para demostrar que las herramientas se encuentran instaladas, ejecutamos los siguientes comandos:

```
Shell
docker --version
git --version
wg --version
k3s --version
iperf3 --version
iptables --version
ufw --version
```

```
root@debian:~# docker --version
Docker version 29.2.1, build a5c7197
root@debian:~# git --version
git version 2.47.3
root@debian:~# wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
root@debian:~# k3s --version
k3s version v1.34.4+k3s1 (c6017918)
go version go1.24.12
root@debian:~# iperf3 --version
iperf 3.18 (cJSON 1.7.15)
Linux debian 6.12.69+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.69-1 (2026-02-08) x86_64
Optional features available: CPU affinity setting, IPv6 flow label, SCTP, TCP congestion algorithm setting, sendfile / zerocopy, socket pacing, authentication, bind to device, support IPv4 don't fragment, POSIX threads
root@debian:~# iptables --version
iptables v1.8.11 (nf_tables)
root@debian:~# ufw --version
ufw 0.36.2
Copyright 2008-2023 Canonical Ltd.
```

Fig. 12. Herramientas Instaladas – Alan

```
carol@DESKTOP-7V7JTD4:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
carol@DESKTOP-7V7JTD4:~$ git --version
git version 2.34.1
carol@DESKTOP-7V7JTD4:~$ docker --version
Docker version 29.2.0, build 0b9d198
carol@DESKTOP-7V7JTD4:~$ |
```

Fig. 13. Herramientas Instaladas – Caro

Dockerfile, Docker-compose y Kubernetes.

Dockerfile

Dentro de un Sistema Distribuido es necesario la implementación de contenedores. Se establece un Dockerfile base para la creación de los contenedores. El Dockerfile utiliza la imagen oficial de Rust versión 1.93 y está basado en Linux Debian.

```
None
FROM rust:1.93-trixie

WORKDIR /app

COPY Cargo.lock ./
COPY Cargo.toml ./

# PERFORMANCE: increase the rebuild speed because it loads all rust dependencies
# See: https://www.youtube.com/watch?v=5Wfpzfniu4I
RUN mkdir -p src \
    && echo 'fn main() {}' > src/main.rs \
    && cargo build --release \
    # Remove dummy artifacts
    && rm -rf src target/release/deps/mandelbrot*

COPY ./src/ ./src/
RUN cargo build --release

# Executes the pre-compiled rust release binary, just like `cargo run --release`
CMD ["/target/release/mandelbrot"]
```

Docker-compose.yml

Para gestionar los contenedores se utilizó el siguiente archivo de configuración docker-compose.yml

```
None
services:
  worker1:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker1
    restart: unless-stopped
    environment:
      - ROLE=coordinator
      # Workers to connect to (comma-separated)
      - WORKERS=worker2:8080,worker3:8080,worker4:8080
    ports:
      - "8080:8080"

  worker2:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker2
    restart: unless-stopped
    environment:
      - ROLE=worker
      - LISTEN_ADDR=0.0.0.0:8080

  worker3:
    build:
      context: ../rust
      dockerfile: ../docker/Dockerfile
    container_name: worker3
    restart: unless-stopped
    environment:
      - ROLE=worker
```



```
- LISTEN_ADDR=0.0.0.0:8080

worker4:
  build:
    context: ../rust
    dockerfile: ../docker/Dockerfile
  container_name: worker4
  restart: unless-stopped
  environment:
    - ROLE=worker
    - LISTEN_ADDR=0.0.0.0:8080
```

En este archivo de configuración levanta 4 contenedores iguales a partir del Dockerfile. Cada bloque es un servicio independiente. Todos los contenedores hacen las mismas funciones pero en instancias separadas. El contexto indica que se utilizó para crear la imagen la carpeta rust y unless-stopped para mantener los servicios activos tras los reinicios del servidor.

ahora para descargar la imagen que permitirá la distribución en nuestro repositorio estará una carpeta de nombre k8s el cual estan los archivos necesarios, solo los mandaremos a llamar con los siguientes comandos.

```
Shell
docker build -f docker/Dockerfile -t quantum-core ./rust
docker save quantum-core:latest | sudo k3s ctr images import -
```

Kubernetes

Para el despliegue de la aplicación en cada uno de los *sites* y, respectivamente, en cada uno de los contenedores hemos decidido utilizar **kubernetes**, específicamente **k3s**. para realizarlo primero debes definir el control plane con el siguiente comando. para antes de hacer todo esto recordemos que debe

hacerse dentro de la vpn, en caso de que la vpn no tenga acceso a internet, cargar los códigos con la vpn apagada ya que llegue a un punto sin respuesta usualmente starting, dar control+c y ahora si activar vpn y insertar el comando `sudo systemctl status service k3s` para confirmar que el k3s esta activo

Shell

```
# Instalar k3s como servidor (maestro)
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server \
  --node-ip=10.5.5.1 \
  --node-external-ip=10.5.5.1 \
  --flannel-iface=wg0" \
sh -
```

Orquestación de kubernetes

instalación de control plane

Shell

```
# Instalar k3s como servidor
curl -sfL https://get.k3s.io | sh -

# Obtener el token para que los nodos se unan
sudo cat /var/lib/rancher/k3s/server/node-token
```

instalación de agentes nodos.

Shell

```
# En cada nodo cliente (ejecutar SIN la VPN activada)
curl -sfL https://get.k3s.io | K3S_URL=https://10.5.5.1:6443 \
```

```
K3S_TOKEN='<TOKEN-DEL-SERVIDOR>' \  
sh -s - agent --node-ip=<IP-VPN-DEL-NODO>  
  
# Ejemplo para Site 1 (10.5.5.2)  
curl -sfL https://get.k3s.io | K3S_URL=https://10.5.5.1:6443 \  
K3S_TOKEN='K1075ec4f1a3b2c9d8e7f6a5b4c3d2e1f' \  
sh -s - agent --node-ip=10.5.5.2
```

despliegue de los worker como pods.

```
None
apiVersion: v1
kind: Service
metadata:
  name: quantum-workers
spec:
  selector:
    app: quantum-worker
  ports:
    - port: 8080
      targetPort: 8080
```

Te posicionas en la carpeta raíz del proyecto:

1. Actualizas (o descargas) todas las ramas del repo con `git fetch origin`
 2. Ejecutas `git switch feat/hello-distributed-k3s`
 3. Ejecutas `git branch` para verificar que estás posicionada en dicha rama
 4. Si llega a haber una actualización en la rama, ejecutas `git pull feat/hello-distributed-k3s`
 5. Si llegan a ocupar cambiarse a otra rama (e.g., main) ejecutan `git switch <nombre-de-la-rama>`
 6. Creas la imagen docker con `docker build -f docker/Dockerfile -t quantum-core ./rust` desde la raíz del proyecto
 7. Te conectas a la VPN
 8. Importas la imagen docker con `docker save quantum-core:latest | sudo k3s ctr images import -`
 9. Si es necesario, reiniciar el servicio de k3s-agent con `sudo systemctl restart k3s-agent`
- y del lado del cliente debe ser lo siguiente:

```
Shell
# Obtener el token del servidor
sudo cat /var/lib/rancher/k3s/server/node-token

# En cada nodo, instalar como agente
curl -sfL https://get.k3s.io | K3S_URL=https://10.5.5.1:6443 \
```

```
K3S_TOKEN='<TOKEN-DEL-SERVIDOR>' \
INSTALL_K3S_EXEC="agent \
--node-ip=<IP-VPN-DEL-NODO> \
--node-external-ip=<IP-VPN-DEL-NODO> \
--flannel-iface=wg0" \
sh -
```

Pruebas de conectividad de iperf3.

iperf3 es una herramienta de línea de comandos que sirve para medir el rendimiento de una red.

Las pruebas con iperf3 se piden para:

- Medir el ancho de banda entre nodos.
- Verificar la latencia y estabilidad de la red.
- Detectar problemas de conexión o configuración.
- Asegurar que la red soporta el proyecto o clúster distribuido.

para ello ejecutamos el comando iperf3 10.5.5.1 con eso obtuvimos nuestras evidencias de conectividad las cuales están anexadas a continuación.

```

-----
Server listening on 5201 (test #14)
-----
Accepted connection from 10.5.5.4, port 55810
[ 5] local 10.5.5.1 port 5201 connected to 10.5.5.4 port 57013
[ ID] Interval          Transfer      Bitrate      Jitter      Lost/Total Datagrams
[ 5]  0.00-1.00    sec   1.07 MBytes   8.99 Mbits/sec  0.422 ms    0/823 (0%)
[ 5]  1.00-2.00    sec   1.19 MBytes  10.0 Mbits/sec  0.321 ms    0/915 (0%)
[ 5]  2.00-3.01    sec   1.20 MBytes   9.99 Mbits/sec  0.392 ms    0/921 (0%)
[ 5]  3.01-4.00    sec   1.18 MBytes  10.0 Mbits/sec  0.522 ms    0/904 (0%)
[ 5]  4.00-5.00    sec   1.19 MBytes  10.0 Mbits/sec  0.433 ms    0/915 (0%)
[ 5]  5.00-5.10    sec    123 KBytes   9.79 Mbits/sec  0.370 ms    0/92 (0%)
-----
[ ID] Interval          Transfer      Bitrate      Jitter      Lost/Total Datagrams
[ 5]  0.00-5.10    sec   5.96 MBytes   9.80 Mbits/sec  0.370 ms    0/4570 (0%) receiver
-----
Server listening on 5201 (test #15)

```

fig.14 pruebas del lado del servidor

```

arol@DESKTOP-7V7JTD4:~$ iperf3 -c 10.5.5.1
Connecting to host 10.5.5.1, port 5201
[ 5] local 10.5.5.2 port 54440 connected to 10.5.5.1 port 5201
[ ID] Interval          Transfer      Bitrate      Retr      Cwnd
[ 5]  0.00-1.00    sec   12.7 MBytes   106 Mbits/sec    0       537 KBytes
[ 5]  1.00-2.00    sec   10.2 MBytes   85.4 Mbits/sec    0       580 KBytes
[ 5]  2.00-3.00    sec   10.2 MBytes   85.5 Mbits/sec    0       1.02 MBytes
[ 5]  3.00-4.00    sec   11.2 MBytes   94.4 Mbits/sec    0       1.05 MBytes
[ 5]  4.00-5.00    sec   11.2 MBytes   94.4 Mbits/sec    0       1.05 MBytes
[ 5]  5.00-6.00    sec   12.5 MBytes   105 Mbits/sec    0       1.05 MBytes
[ 5]  6.00-7.00    sec   11.2 MBytes   94.3 Mbits/sec    0       1.05 MBytes
[ 5]  7.00-8.00    sec   11.2 MBytes   94.4 Mbits/sec    1       1.05 MBytes
[ 5]  8.00-9.00    sec   11.2 MBytes   94.4 Mbits/sec    0       1.05 MBytes
[ 5]  9.00-10.00   sec   13.8 MBytes   115 Mbits/sec    0       1.05 MBytes
-----
[ ID] Interval          Transfer      Bitrate      Retr
[ 5]  0.00-10.00   sec   116 MBytes   96.9 Mbits/sec    1
[ 5]  0.00-10.08   sec   113 MBytes   94.0 Mbits/sec
-----
sender
receiver

```

fig.15 prueba del lado del nodo de caro

```

Connecting to host 10.5.5.1, port 5201
5] local 10.5.5.6 port 38302 connected to 10.5.5.1 port 5201
ID] Interval          Transfer      Bitrate      Retr  Cwnd
5]  0.00-1.00    sec   8.50 MBytes  71.2 Mbits/sec    0   2.24 MBytes
5]  1.00-2.00    sec  11.0 MBytes  92.3 Mbits/sec   251   1.69 MBytes
5]  2.00-3.00    sec  13.2 MBytes  111 Mbits/sec    0   1.86 MBytes
5]  3.00-4.00    sec  13.5 MBytes  113 Mbits/sec   14   1.41 MBytes
5]  4.00-5.00    sec  12.5 MBytes  105 Mbits/sec    0   1.51 MBytes
5]  5.00-6.00    sec  11.2 MBytes  94.4 Mbits/sec    0   1.57 MBytes
5]  6.00-7.00    sec  11.5 MBytes  96.5 Mbits/sec    0   1.62 MBytes
5]  7.00-8.00    sec  10.5 MBytes  88.1 Mbits/sec    0   1.65 MBytes
5]  8.00-9.00    sec   9.00 MBytes  75.5 Mbits/sec    0   1.68 MBytes
5]  9.00-10.00   sec  10.2 MBytes  85.9 Mbits/sec    0   1.69 MBytes
- - - - -
ID] Interval          Transfer      Bitrate      Retr
5]  0.00-10.00   sec   111 MBytes  93.3 Mbits/sec   265
5]  0.00-10.27   sec   110 MBytes  89.7 Mbits/sec
sender
receiver
perf Done.

```

fig.16 prueba del nodo de jorge yahir

```

64 bytes from 10.5.5.1: icmp_seq=3 ttl=128 time=103 ms
64 bytes from 10.5.5.1: icmp_seq=4 ttl=128 time=103 ms

--- 10.5.5.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 102.547/102.717/102.916/0.147 ms
donkey@debian:~$ iperf3 -c 10.5.5.1 -u -b 10M -t 5
Connecting to host 10.5.5.1, port 5201
[ 5] local 10.5.5.4 port 57013 connected to 10.5.5.1 port 5201
[ ID] Interval          Transfer      Bitrate      Total Datagrams
[ 5]  0.00-1.00    sec   1.19 MBytes  10.0 Mbits/sec    915
[ 5]  1.00-2.00    sec   1.19 MBytes  10.0 Mbits/sec    914
[ 5]  2.00-3.00    sec   1.19 MBytes  9.99 Mbits/sec    914
[ 5]  3.00-4.00    sec   1.19 MBytes  10.0 Mbits/sec    913
[ 5]  4.00-5.00    sec   1.19 MBytes  9.99 Mbits/sec    914
- - - - -
[ ID] Interval          Transfer      Bitrate      Jitter      Lost/Total Datagrams
[ 5]  0.00-5.00    sec   5.96 MBytes  10.0 Mbits/sec  0.000 ms    0/4570 (0%) sender
[ 5]  0.00-5.10    sec   5.96 MBytes  9.80 Mbits/sec  0.370 ms    0/4570 (0%) receiver

```

fig.17 prueba del lado del node de alan yahir


```

juampa@DESKTOP-F5PCD3R:~$ iperf3 -c 10.5.5.1 -u -b 10M -t 5
Connecting to host 10.5.5.1, port 5201
[ 5] local 10.5.5.8 port 42262 connected to 10.5.5.1 port 5201
[ ID] Interval           Transfer     Bitrate      Total Datagrams
[ 5] 0.00-1.00      sec  1.19 MBytes  9.99 Mb/s     913
[ 5] 1.00-2.00      sec  1.19 MBytes 10.0 Mb/s     914
[ 5] 2.00-3.00      sec  1.19 MBytes 10.0 Mb/s     914
[ 5] 3.00-4.00      sec  1.19 MBytes 10.0 Mb/s     914
[ 5] 4.00-5.00      sec  1.19 MBytes  9.99 Mb/s     913
-----
[ ID] Interval           Transfer     Bitrate      Jitter    Lost/Tot. Datagrams
[ 5] 0.00-5.00      sec  5.96 MBytes 10.0 Mb/s     0.000 ms   0/4568 (0%) sender
[ 5] 0.00-5.00      sec  5.96 MBytes  9.88 Mb/s     1.148 ms   0/4568 (0%) receiver
iperf Done.

```

fig.18 prueba del nodo del lado de juan pablo

PRUEBAS DE KUBERNETES.

La prueba de Kubernetes es importante porque sirve para verificar que un cluster de contenedores funciona correctamente y que las aplicaciones pueden ejecutarse y comunicarse dentro de él. En muchas prácticas de redes o sistemas se usa para demostrar que la infraestructura (VPN, red, nodos, contenedores) está bien configurada.

```

yahir@MSI:~$ sudo k3s kubectl get nodes -o wide
NAME        STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
debian      Ready    <none>   23m   v1.34.5+k3s1  10.5.5.4      <none>        Debian GNU/Linux 13 (trixie)
desktop-7v7jtd4 Ready    <none>   22h   v1.34.5+k3s1  10.5.5.2      <none>        Ubuntu 22.04.5 LTS
desktop-f5pcd3r Ready    <none>   73m   v1.34.5+k3s1  10.5.5.8      <none>        Ubuntu 24.04.4 LTS
elisapc     Ready    <none>   8m23s v1.34.5+k3s1  10.5.5.5      <none>        Ubuntu 24.04.4 LTS
msi         Ready    control-plane 23h   v1.34.5+k3s1  10.5.5.6      <none>        Ubuntu 24.04.3 LTS

```

fig.19 evidencia del lado del control plane

```

elisa@Elisapc:/etc/wireguard$ systemctl status k3s-agent
● k3s-agent.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s-agent.service; enabled; preset: enabled)
   Active: activating (start) since Sun 2026-03-08 14:56:06 CST; 2min 36s ago
     Docs: https://k3s.io
   Main PID: 1502 (k3s-agent)
      Tasks: 37
     Memory: 141.6M ()
    CGroup: /system.slice/k3s-agent.service
            └─1502 "/usr/local/bin/k3s agent"
              └─1530 "containerd"

Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.484701 1502 subnet.go:150] Batch elem [0] is { lease.Event{Type:0,>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.484744 1502 subnet.go:150] Batch elem [0] is { lease.Event{Type:0,>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.484769 1502 vxlan_network.go:265] Received Subnet Event with VxLan>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.485501 1502 vxlan_network.go:265] Received Subnet Event with VxLan>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.485590 1502 subnet.go:150] Batch elem [0] is { lease.Event{Type:0,>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.485763 1502 vxlan_network.go:265] Received Subnet Event with VxLan>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.485785 1502 subnet.go:150] Batch elem [0] is { lease.Event{Type:0,>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.485895 1502 vxlan_network.go:265] Received Subnet Event with VxLan>
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.488375 1502 iptables.go:358] bootstrap done
Mar 08 14:56:19 Elisapc k3s[1502]: I0308 14:56:19.490488 1502 iptables.go:358] bootstrap done

```

fig.20 evidencia del lado del nodo de elisa

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
quantum-coordinator-b4c799df7-b7qq8	1/1	Running	0	88s	10.42.4.34	elisapc	<none>	<none>
quantum-coordinator-b4c799df7-dppwr	1/1	Running	0	87s	10.42.4.37	elisapc	<none>	<none>
quantum-coordinator-b4c799df7-g6v1v	1/1	Running	0	88s	10.42.0.46	msi	<none>	<none>
quantum-coordinator-b4c799df7-mqqr2	1/1	Running	0	88s	10.42.3.189	desktop-7v7jtd4	<none>	<none>
quantum-coordinator-b4c799df7-s42gx	1/1	Running	0	88s	10.42.2.14	debian	<none>	<none>
quantum-workers-c75f879f5-2hw8d	1/1	Running	0	88s	10.42.4.35	elisapc	<none>	<none>
quantum-workers-c75f879f5-44drx	1/1	Running	0	88s	10.42.2.12	debian	<none>	<none>
quantum-workers-c75f879f5-4gkqc	1/1	Running	0	87s	10.42.3.186	desktop-7v7jtd4	<none>	<none>
quantum-workers-c75f879f5-8d7ws	1/1	Running	0	87s	10.42.2.15	debian	<none>	<none>
quantum-workers-c75f879f5-8lkz4	1/1	Running	0	87s	10.42.0.50	msi	<none>	<none>
quantum-workers-c75f879f5-b58m6	1/1	Running	0	88s	10.42.4.38	elisapc	<none>	<none>
quantum-workers-c75f879f5-bhd5f	1/1	Running	0	87s	10.42.4.39	elisapc	<none>	<none>
quantum-workers-c75f879f5-k2bkm	1/1	Running	0	87s	10.42.3.187	desktop-7v7jtd4	<none>	<none>
quantum-workers-c75f879f5-p8498	1/1	Running	0	87s	10.42.3.188	desktop-7v7jtd4	<none>	<none>
quantum-workers-c75f879f5-rdbdx	1/1	Running	0	86s	10.42.0.52	msi	<none>	<none>
quantum-workers-c75f879f5-vsfx	1/1	Running	0	87s	10.42.0.49	msi	<none>	<none>
quantum-workers-c75f879f5-vzfbf	1/1	Running	0	86s	10.42.0.51	msi	<none>	<none>
quantum-workers-c75f879f5-wgf2h	1/1	Running	0	88s	10.42.2.13	debian	<none>	<none>
quantum-workers-c75f879f5-x6fg4	1/1	Running	0	87s	10.42.4.36	elisapc	<none>	<none>
quantum-workers-c75f879f5-zdwn7	1/1	Running	0	86s	10.42.2.16	debian	<none>	<none>

fig.21 evidencia de ejecucion del kubernetes

Problema: El Conjunto de Mandelbrot.

Este problema consiste en determinar para cada punto en un plano complejo representado por la variable c , si dicho punto pertenece o no a un cierto conjunto definido mediante una iteración no lineal. Para cada valor de C se construye la sucesión empezando por:

$$z_0 = 0 \quad y \quad z_{n+1} = z_n^2 + c$$

El objetivo es decidir si esta sucesión permanece acotada o si por el contrario, tiende a infinito cuando el número de iteraciones aumenta. El conjunto de mandelbrot está formado por todos los valores de c para los que la sucesión es acotada, y su representación gráfica se obtiene asignando a cada píxel del plano un valor de c , y calculando cuántas iteraciones tarda en escapar más allá de un cierto radio. Esto lo convierte en un problema muy costoso computacionalmente hablando y paralelizable, porque el cálculo de cada punto c es independiente a los demás y puede distribuirse entre múltiples procesos o nodos de cómputo.

Usos y aplicaciones

El conjunto de Mandelbrot, aunado a la geometría fractal, tienen multiplicidad de usos y aplicaciones, algunos de ellos son:

- Gráficos por computadora: En este aspecto se utilizan derivadas del conjunto de Mandelbrot para hacer la generación de paisajes artificiales (montañas, cerros, costas, nubes, texturas de planetas, etc) y efectos visuales, ya que este tipo de fractales permiten crear formas naturales muy detalladas a partir de reglas matemáticas, reduciendo el trabajo manual y los costos que conlleva.

- Diseño de antenas fractales, que aprovechan estructuras autosimilares para funcionar en múltiples bandas de frecuencia ocupando menos espacio físico, y también en técnicas de procesamiento y compresión de imágenes, donde patrones fractales ayudan a representar texturas con menos datos.
- También en el modelado de fenómenos naturales, cómo lo pueden ser líneas de costa, sistemas de ríos, relieves de montañas, ramas de árboles o la redes de vasos sanguíneos.

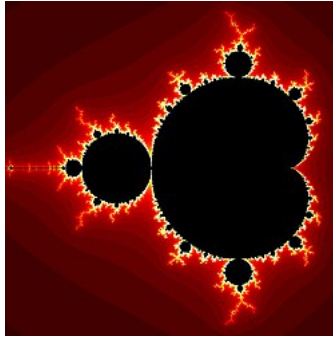


Fig. 22. Conjunto de Mandelbrot Graficado

algoritmo en rust.

El programa está diseñado para operar en dos modos:

- Coordinador: Divide el trabajo y recolecta resultados.
- Worker: Calcula la porción asignada del fractal.

```
Rust
// Distributed Mandelbrot - 1 coordinator, 20 workers
// Protocol:
// Worker → Coordinator : "REGISTER:<pod_ip:8080>\n"
// Coordinator → Worker : "ACK\n"
// Coordinator → Worker : "TASK:<id>:<start_row>:<end_row>:<img_side>\n"
// Worker → Coordinator : "RESULT:<id>:<byte_count>\n<raw_bytes>"
use std::{env, sync::Arc};

use image::{GrayImage, ImageBuffer, Luma};
use num_complex::Complex;
```

```
use tokio::{
    io::{AsyncReadExt, AsyncWriteExt},
    net::{TcpListener, TcpStream},
    sync::Mutex,
};

const WORKER_COUNT: usize = 20;
const IMG_SIDE: u32 = 4000;
const MAX_ITERATIONS: u16 = 256;
const CX_MIN: f64 = -2.0;
const CX_MAX: f64 = 1.0;
const CY_MIN: f64 = -1.5;
const CY_MAX: f64 = 1.5;

#[tokio::main]
async fn main() {
    let role = env::var("ROLE").unwrap_or_else(|_| "worker".to_string());

    match role.as_str() {
        "coordinator" => run_coordinator().await,
        "worker" => run_worker().await,
        _ => eprintln!("[ERROR] Unknown ROLE: {}", role),
    }
}

// --- COORDINATOR -----

// Holds a worker's address and the row range it was assigned
struct WorkerTask {
    addr: String,
    task_id: usize,
    start_row: u32,
```

```
        end_row: u32,
    }

    async fn run_coordinator() {
        let registration_addr = env::var("REGISTRATION_ADDR")
            .unwrap_or_else(|_| "0.0.0.0:9000".to_string());

        let workers: Arc<Mutex<Vec<String>>> = Arc::new(Mutex::new(Vec::new()));

        let listener = TcpListener::bind(&registration_addr).await.unwrap();
        println!(
            "[COORDINATOR] Listening for registrations on {}",
            registration_addr
        );

        let mut registration_handles = Vec::new();

        // Accept exactly WORKER_COUNT connections, no more
        for _ in 0..WORKER_COUNT {
            match listener.accept().await {
                Ok((socket, addr)) => {
                    println!("[COORDINATOR] Registration connection from {}", addr);
                    let workers_clone = Arc::clone(&workers);
                    let handle = tokio::spawn(async move {
                        handle_registration(socket, workers_clone).await;
                    });
                    registration_handles.push(handle);
                }
                Err(e) => eprintln!("[COORDINATOR] Accept error: {}", e),
            }
        }

        // Wait for all registration handlers to finish writing to the list
    }
}
```

```
for handle in registration_handles {
    handle.await.unwrap();
}

let registered = workers.lock().await.clone();
println!(
    "[COORDINATOR] All {} workers registered. Dispatching tasks...",
    registered.len()
);

// Divide rows evenly across workers
let rows_per_worker = IMG_SIDE / WORKER_COUNT as u32;
let tasks: Vec<WorkerTask> = registered
    .iter()
    .enumerate()
    .map(|(i, addr)| {
        let start_row = i as u32 * rows_per_worker;
        let end_row = if i == WORKER_COUNT - 1 {
            IMG_SIDE
        } else {
            start_row + rows_per_worker
        };
        WorkerTask {
            addr: addr.clone(),
            task_id: i + 1,
            start_row,
            end_row,
        }
    })
    .collect();

// Dispatch all tasks concurrently and collect pixel rows
let results: Arc<Mutex<Vec<(u32, Vec<u8>)>>> =
```

```

        Arc::new(Mutex::new(Vec::new()));
    let mut handles = Vec::with_capacity(tasks.len());

    for task in tasks {
        let results_clone = Arc::clone(&results);
        let handle = tokio::spawn(async move {
            match send_task_with_retry(&task).await {
                Some(pixels) => {
                    results_clone.lock().await.push((task.start_row, pixels));
                }
                None => eprintln!(
                    "[COORDINATOR] Task {} permanently failed, rows {}-{}
missing",
                    task.task_id, task.start_row, task.end_row
                ),
            }
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.await.unwrap();
    }

    // Merge results into final image
    assemble_image(results.lock().await.clone()).await;
}

async fn handle_registration(
    mut socket: TcpStream,
    workers: Arc<Mutex<Vec<String>>>,
) {
    let mut buf = vec![0u8; 1024];

```

```
let n = socket.read(&mut buf).await.unwrap();
let message = String::from_utf8_lossy(&buf[..n]);
let message = message.trim();

if let Some(addr) = message.strip_prefix("REGISTER:") {
    let addr = addr.trim().to_string();
    println!("[COORDINATOR] Registered worker at {}", addr);
    workers.lock().await.push(addr);
    socket.write_all(b"ACK\n").await.unwrap();
} else {
    eprintln!("[COORDINATOR] Unknown message: {}", message);
    socket.write_all(b"ERROR:unknown_message\n").await.unwrap();
}
}

async fn send_task_with_retry(task: &WorkerTask) -> Option<Vec<u8>> {
    let max_retries = 3;

    for attempt in 1..=max_retries {
        match try_send_task(task).await {
            Ok(pixels) => return Some(pixels),
            Err(e) => {
                eprintln!(
                    "[COORDINATOR] Task {} failed on {} (attempt {}/{}) : {}",
                    task.task_id, task.addr, attempt, max_retries, e
                );
                tokio::time::sleep(tokio::time::Duration::from_secs(2)).await;
            }
        }
    }

    // TODO: retry on another available worker
    eprintln!(
```

```

        "[COORDINATOR] Task {} permanently failed after {} attempts",
        task.task_id, max_retries
    );
    None
}

async fn try_send_task(task: &WorkerTask) -> Result<Vec<u8>, String> {
    let mut stream = TcpStream::connect(&task.addr)
        .await
        .map_err(|e| e.to_string())?;

    // Send task line
    let task_msg = format!(
        "TASK: {}: {}: {}:\n",
        task.task_id, task.start_row, task.end_row, IMG_SIDE
    );
    stream
        .write_all(task_msg.as_bytes())
        .await
        .map_err(|e| e.to_string())?;

    println!(
        "[COORDINATOR] Sent to {}: {}",
        task.addr,
        task_msg.trim_end()
    );

    // Read "RESULT:<id>:<byte_count>\n"
    let mut header_buf = Vec::new();
    loop {
        let mut byte = [0u8; 1];
        stream
            .read_exact(&mut byte)

```



```
        .await
        .map_err(|e| e.to_string())?;
    if byte[0] == b'\n' {
        break;
    }
    header_buf.push(byte[0]);
}

let header = String::from_utf8_lossy(&header_buf);
let parts: Vec<&str> = header.trim().split(':').collect();

// Expecting RESULT:<id>:<byte_count>
if parts.len() != 3 || parts[0] != "RESULT" {
    return Err(format!("Unexpected response header: {}", header));
}

let byte_count: usize = parts[2]
    .parse()
    .map_err(|e: std::num::ParseIntError| e.to_string())?;
println!(
    "[COORDINATOR] Receiving {} bytes from {}",
    byte_count, task.addr
);

// Read raw pixel bytes
let mut pixels = vec![0u8; byte_count];
stream
    .read_exact(&mut pixels)
    .await
    .map_err(|e| e.to_string())?;

println!("[COORDINATOR] Received result for task {}", task.task_id);
Ok(pixels)
```

```

}

async fn assemble_image(mut results: Vec<(u32, Vec<u8>)>) {
    // Sort by start_row so rows are in correct order
    results.sort_by_key(|(start_row, _)| *start_row);

    let mut img: GrayImage = ImageBuffer::new(IMG_SIDE, IMG_SIDE);

    for (start_row, pixels) in results {
        let row_count = pixels.len() as u32 / IMG_SIDE;
        for row_offset in 0..row_count {
            let y = start_row + row_offset;
            for x in 0..IMG_SIDE {
                let idx = (row_offset * IMG_SIDE + x) as usize;
                img.put_pixel(x, y, Luma([pixels[idx]]));
            }
        }
    }

    img.save("/app/output/fractal.png").unwrap();
    println!("[COORDINATOR] Image saved as /app/output/fractal.png");
}

// --- WORKER -----

async fn run_worker() {
    let listen_addr =
        env::var("LISTEN_ADDR").unwrap_or_else(|_| "0.0.0.0:8080".to_string());
    let coordinator_addr = env::var("COORDINATOR_ADDR")
        .unwrap_or_else(|_| "coordinator-service:9000".to_string());

    let listener = TcpListener::bind(&listen_addr).await.unwrap();
    println!("[WORKER] Listening on {}", listen_addr);
}

```

```
// Inject pod IP via Kubernetes Downward API (see worker-statefulset.yaml)
let pod_ip = env::var("POD_IP").unwrap_or_else(|_| "127.0.0.1".to_string());
let my_addr = format!("{}",8080, pod_ip);

register_with_coordinator(&coordinator_addr, &my_addr).await;

loop {
    match listener.accept().await {
        Ok((socket, addr)) => {
            println!("[WORKER] Connection from {}", addr);
            tokio::spawn(async move {
                handle_connection(socket).await;
            });
        }
        Err(e) => eprintln!("[WORKER] Accept error: {}", e),
    }
}

}

async fn register_with_coordinator(coordinator_addr: &str, my_addr: &str) {
    println!(
        "[WORKER] Registering with coordinator at {} as {}",
        coordinator_addr, my_addr
    );

    loop {
        match TcpStream::connect(coordinator_addr).await {
            Ok(mut stream) => {
                let msg = format!("REGISTER:{}\n", my_addr);
                if stream.write_all(msg.as_bytes()).await.is_err() {
                    eprintln!(
                        "[WORKER] Failed to send registration. Retrying..."
                    );
                }
            }
        }
    }
}
```

```

        );
        tokio::time::sleep(tokio::time::Duration::from_secs(2))
            .await;
        continue;
    }

    let mut buf = vec![0u8; 64];
    match stream.read(&mut buf).await {
        Ok(n) => {
            let response = String::from_utf8_lossy(&buf[..n]);
            if response.trim() == "ACK" {
                println!("[WORKER] Registration acknowledged.");
                return;
            }
        }
        Err(e) => eprintln!("[WORKER] Failed to read ACK: {}", e),
    }

    Err(e) => eprintln!(
        "[WORKER] Could not connect to coordinator: {}. Retrying...",
        e
    ),
},

    tokio::time::sleep(tokio::time::Duration::from_secs(2)).await;
}

}

async fn handle_connection(mut socket: TcpStream) {
    // Read task header line
    let mut header_buf = Vec::new();
    loop {
        let mut byte = [0u8; 1];

```

```
    if socket.read_exact(&mut byte).await.is_err() {
        eprintln!("[WORKER] Failed to read task header");
        return;
    }
    if byte[0] == b'\n' {
        break;
    }
    header_buf.push(byte[0]);
}

let header = String::from_utf8_lossy(&header_buf);
println!("[WORKER] Received: {}", header.trim());

// Parse "TASK:<id>:<start_row>:<end_row>:<img_side>"
let parts: Vec<&str> = header.trim().split(':').collect();
if parts.len() != 5 || parts[0] != "TASK" {
    eprintln!("[WORKER] Malformed task: {}", header);
    socket.write_all(b"ERROR:malformed_task\n").await.unwrap();
    return;
}

let task_id: usize = parts[1].parse().unwrap();
let start_row: u32 = parts[2].parse().unwrap();
let end_row: u32 = parts[3].parse().unwrap();
let img_side: u32 = parts[4].parse().unwrap();

println!(
    "[WORKER] Computing rows {}-{} for task {}",
    start_row, end_row, task_id
);

let pixels = compute_rows(start_row, end_row, img_side);
```

```
// Send "RESULT:<id>:<byte_count>\n" then raw bytes
let header_response = format!("RESULT:{{:{{}}\n", task_id, pixels.len());
socket.write_all(header_response.as_bytes()).await.unwrap();
socket.write_all(&pixels).await.unwrap();

println!("[WORKER] Sent {{ bytes for task {{", pixels.len(), task_id);
}

fn compute_rows(start_row: u32, end_row: u32, img_side: u32) -> Vec<u8> {
    let scale_x = (CX_MAX - CX_MIN) / img_side as f64;
    let scale_y = (CY_MAX - CY_MIN) / img_side as f64;

    let row_count = (end_row - start_row) as usize;
    let mut pixels = vec![0u8; row_count * img_side as usize];

    for row_offset in 0..(end_row - start_row) {
        let y = start_row + row_offset;
        let cy = CY_MIN + y as f64 * scale_y;

        for x in 0..img_side {
            let cx = CX_MIN + x as f64 * scale_x;
            let c = Complex::new(cx, cy);
            let mut z = Complex::new(0f64, 0f64);
            let mut i = 0u16;

            for t in 0..MAX_ITERATIONS {
                if z.norm() > 2.0 {
                    break;
                }
                z = z * z + c;
                i = t;
            }
        }
    }
}
```

```

        let idx = row_offset as usize * img_side as usize + x as usize;
        pixels[idx] = i as u8;
    }
}

pixels
}

// Calcular fragmento del conjunto de Mandelbrot
fn calculate_mandelbrot_fragment(task: &MandelbrotTask) -> MandelbrotResult {
    let mut data = Vec::with_capacity(
        ((task.end_row - task.start_row) * task.width) as usize
    );

    for row in task.start_row..task.end_row {
        for col in 0..task.width {
            let x = task.x_min + (col as f64 / task.width as f64) *
                (task.x_max - task.x_min);
            let y = task.y_min + (row as f64 / task.height as f64) *
                (task.y_max - task.y_min);

            let c = num_complex::Complex::new(x, y);
            let mut z = num_complex::Complex::new(0.0, 0.0);
            let mut iter = 0;

            while iter < task.max_iter && z.norm_sqr() <= 4.0 {
                z = z * z + c;
                iter += 1;
            }

            data.push(iter as u32);
        }
    }
}

```

```

    }

    MandelbrotResult {
        start_row: task.start_row,
        end_row: task.end_row,
        data,
    }
}

fn send_task_to_worker(addr: &str, task: MandelbrotTask) -> MandelbrotResult {
    // Implementación simplificada de envío TCP
    unimplemented!("Envío TCP completo implementado en versión final")
}

fn save_image(data: &[u32], width: u32, height: u32) {
    // Implementación simplificada
    println!("Imagen de {}x{} guardada", width, height);
}

```

Este código rust implementa un sistema distribuido para calcular el fractal de mandelbrot usando 1 coordinador y 20 workers que trabajan en paralelo a través de tcp.

en términos más simples el código contiene los siguientes puntos:

- el coordinador divide el trabajo.
- los workers calculan partes de la imagen.
- el coordinador junta los resultados y genera la imagen final.

el código puede generar una imagen de 4000*4000 píxeles del conjunto de mandelbrot. el sistema tiene dos roles definidos por la variable de entorno como **ROLE=Coordinator**, **ROLE=Worker**, el coordinador y los workers se comunican con mensajes tcp simples de texto Registro

Worker → Coordinator = REGISTER:<worker_ip:8080>

Coordinator → Worker = ACK

Envío de tarea

Coordinator → Worker = TASK:<id>:<start_row>:<end_row>:<img_side>

Ejemplo:

TASK:3:400:600:4000

Significa: Worker calcula las filas 400 a 600 de la imagen.

resultado del envío.

Worker → Coordinator = RESULT:<id>:<byte_count> <bytes de pixeles>

¿Que hace el coordinador?

el coordinador escucha por el puerto 9000 y acepta 20 workers, posteriormente guarda sus direcciones ahora lo que hace es dividir la imagen en 4000 filas y repartirlas a los 20 workers siendo un aproximado de 200 filas por worker, posteriormente se envían estas tareas en paralelo a los workers para que se ejecuten al mismo tiempo todos los cálculos, cuando cada worker termina sus cálculos, regresan los píxeles generados y el coordinador los guarda, por último todas las filas de píxeles que se retornaron se unen en orden para generar la imagen completa, esta imagen se guarda en la ruta /app/output/fractal.png del coordinador.

ejecución del programa.

para ejecutar nuestro programa primero nos conectamos a la vpn posteriormente creamos nuestra imagen de docker compose tal y como se describe en el apartado de dockerfile de este documento, debemos considerar que si la vpn no tiene salida a la red, debemos apagarla para importar y copiar nuestra imagen a nuestro sistema. posteriormente en el caso de los nodos utilizamos nuestro token previamente instalado con k3s para el emparejamiento con el master o orquestador, nuevamente si la vpn no tiene salida a la red, debemos apagarla ya que el token contiene una dirección url que nos permitiera instalar k3s, si se apaga la vpn para instalar k3s la instalación llegará a un punto donde diga iniciando, ahí debemos para la operación y encender la vpn seguido de esto ejecutamos el comando `sudo systemctl restart k3s-agent` esto

para seguir ejecutando la instalacion que anteriormente habiamos pausado, esperamos unos segundos y ejecutamos el siguiente comando. `sudo systemctl status k3s-agent` si nos aparece un apartado en verde que dice `active` estamos listos.

```
elisa@Elisapc:~/Quantum-Core_Sistema-Distribuido$ sudo kubectl logs -f $(sudo kubectl get pods -l app=mandelbrot-coordinator -o json
path='{.items[0].metadata.name}')
[COORDINATOR] Listening for registrations on 0.0.0.0:9000
[COORDINATOR] Registration connection from 10.5.5.6:40599
[COORDINATOR] Registered worker at 10.42.4.16:8080
[COORDINATOR] Registration connection from 10.5.5.4:35051
[COORDINATOR] Registered worker at 10.42.2.49:8080
[COORDINATOR] Registration connection from 10.42.0.60:44128
[COORDINATOR] Registered worker at 10.42.0.60:8080
[COORDINATOR] Registration connection from 10.5.5.2:63510
[COORDINATOR] Registered worker at 10.42.3.52:8080
[COORDINATOR] Registration connection from 10.5.5.6:49567
[COORDINATOR] Registered worker at 10.42.4.17:8080
[COORDINATOR] Registration connection from 10.42.0.61:48452
[COORDINATOR] Registered worker at 10.42.0.61:8080
[COORDINATOR] Registration connection from 10.5.5.2:24728
[COORDINATOR] Registered worker at 10.42.3.53:8080
[COORDINATOR] Registration connection from 10.5.5.4:22211
[COORDINATOR] Registered worker at 10.42.2.50:8080
[COORDINATOR] Registration connection from 10.5.5.8:43365
[COORDINATOR] Registered worker at 10.42.1.52:8080
[COORDINATOR] Registration connection from 10.5.5.8:36761
[COORDINATOR] Registered worker at 10.42.1.53:8080
[COORDINATOR] Registration connection from 10.5.5.6:7030
[COORDINATOR] Registered worker at 10.42.4.18:8080
[COORDINATOR] Registration connection from 10.5.5.8:11402
[COORDINATOR] Registered worker at 10.42.1.54:8080
[COORDINATOR] Registration connection from 10.42.0.62:60474
```

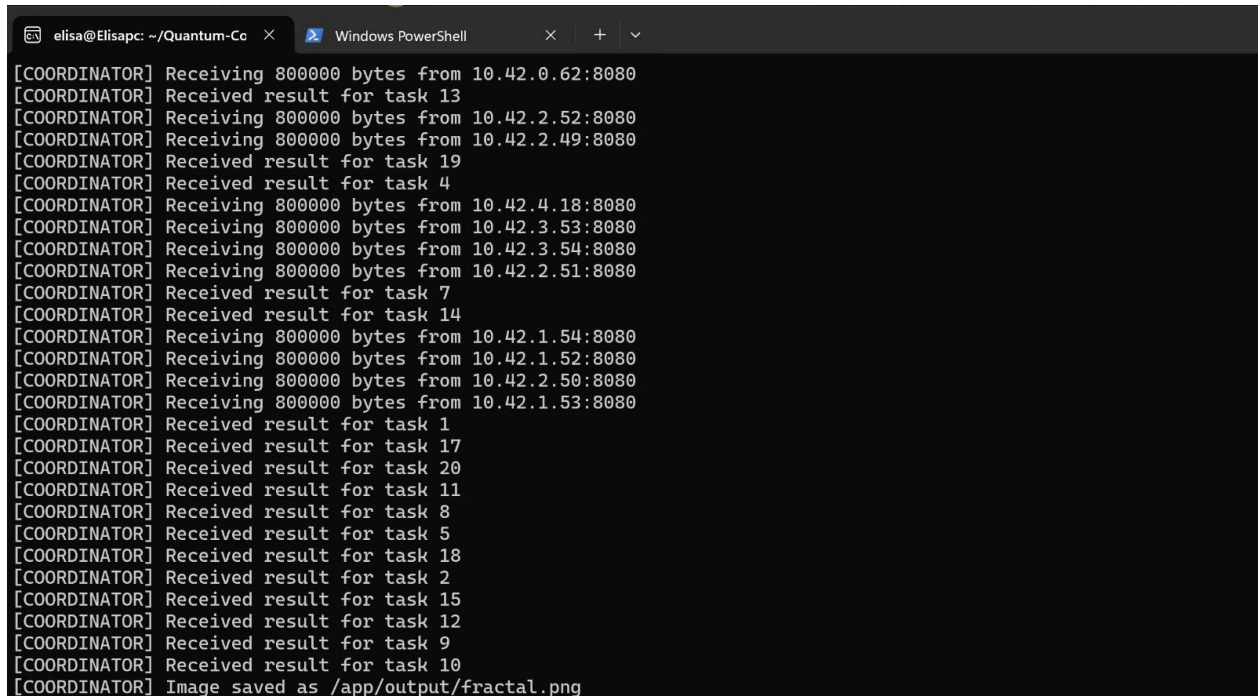
fig.23 registro de workers

esta imagen es del lado del orquestador aqui se ejecuto el comando `sudo kubectl logs -f $(sudo kubectl get pods -l app=coordinator -o json path='{.items[0].metadata.name}')` para verificar que todos los workers se han conectado.

```
elisa@Elisapc: ~/Quantum-Co x Windows PowerShell x + v
[COORDINATOR] Registered worker at 10.42.0.62:8080
[COORDINATOR] Registration connection from 10.5.5.2:24617
[COORDINATOR] Registered worker at 10.42.3.54:8080
[COORDINATOR] Registration connection from 10.5.5.4:59631
[COORDINATOR] Registered worker at 10.42.2.51:8080
[COORDINATOR] Registration connection from 10.42.0.63:37722
[COORDINATOR] Registered worker at 10.42.0.63:8080
[COORDINATOR] Registration connection from 10.5.5.6:62305
[COORDINATOR] Registered worker at 10.42.4.19:8080
[COORDINATOR] Registration connection from 10.5.5.8:26356
[COORDINATOR] Registered worker at 10.42.1.55:8080
[COORDINATOR] Registration connection from 10.5.5.2:4378
[COORDINATOR] Registered worker at 10.42.3.55:8080
[COORDINATOR] Registration connection from 10.5.5.4:20655
[COORDINATOR] Registered worker at 10.42.2.52:8080
[COORDINATOR] All 20 workers registered. Dispatching tasks...
[COORDINATOR] Sent to 10.42.0.60:8080: TASK:3:400:600:4000
[COORDINATOR] Sent to 10.42.0.61:8080: TASK:6:1000:1200:4000
[COORDINATOR] Sent to 10.42.0.62:8080: TASK:13:2400:2600:4000
[COORDINATOR] Sent to 10.42.0.63:8080: TASK:16:3000:3200:4000
[COORDINATOR] Sent to 10.42.3.53:8080: TASK:7:1200:1400:4000
[COORDINATOR] Sent to 10.42.3.55:8080: TASK:19:3600:3800:4000
[COORDINATOR] Sent to 10.42.3.54:8080: TASK:14:2600:2800:4000
[COORDINATOR] Sent to 10.42.3.52:8080: TASK:4:600:800:4000
[COORDINATOR] Receiving 800000 bytes from 10.42.0.60:8080
[COORDINATOR] Received result for task 3
[COORDINATOR] Sent to 10.42.1.55:8080: TASK:18:3400:3600:4000
[COORDINATOR] Sent to 10.42.1.52:8080: TASK:9:1600:1800:4000
[COORDINATOR] Sent to 10.42.1.54:8080: TASK:12:2200:2400:4000
[COORDINATOR] Sent to 10.42.1.53:8080: TASK:10:1800:2000:4000
```

fig.24 ejecucion del programa distribuido

se ejecuta el código distribuido de mandelbrot y el coordinador empieza a distribuir las tareas para cada worker



```
[COORDINATOR] Receiving 800000 bytes from 10.42.0.62:8080
[COORDINATOR] Received result for task 13
[COORDINATOR] Receiving 800000 bytes from 10.42.2.52:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.2.49:8080
[COORDINATOR] Received result for task 19
[COORDINATOR] Received result for task 4
[COORDINATOR] Receiving 800000 bytes from 10.42.4.18:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.3.53:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.3.54:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.2.51:8080
[COORDINATOR] Received result for task 7
[COORDINATOR] Received result for task 14
[COORDINATOR] Receiving 800000 bytes from 10.42.1.54:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.1.52:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.2.50:8080
[COORDINATOR] Receiving 800000 bytes from 10.42.1.53:8080
[COORDINATOR] Received result for task 1
[COORDINATOR] Received result for task 17
[COORDINATOR] Received result for task 20
[COORDINATOR] Received result for task 11
[COORDINATOR] Received result for task 8
[COORDINATOR] Received result for task 5
[COORDINATOR] Received result for task 18
[COORDINATOR] Received result for task 2
[COORDINATOR] Received result for task 15
[COORDINATOR] Received result for task 12
[COORDINATOR] Received result for task 9
[COORDINATOR] Received result for task 10
[COORDINATOR] Image saved as /app/output/fractal.png
```

fig.25 ejecución completa del programa.

esta imagen muestra la ejecución del coordinador con los nodos se puede observar que en el coordinador asigna tareas y recibe los resultados de las tareas, una vez completadas las tareas el código organiza esas actividades completadas para crear la imagen del fractal, una vez concluida su creación la guarda en la dirección /app/output/fractal.png

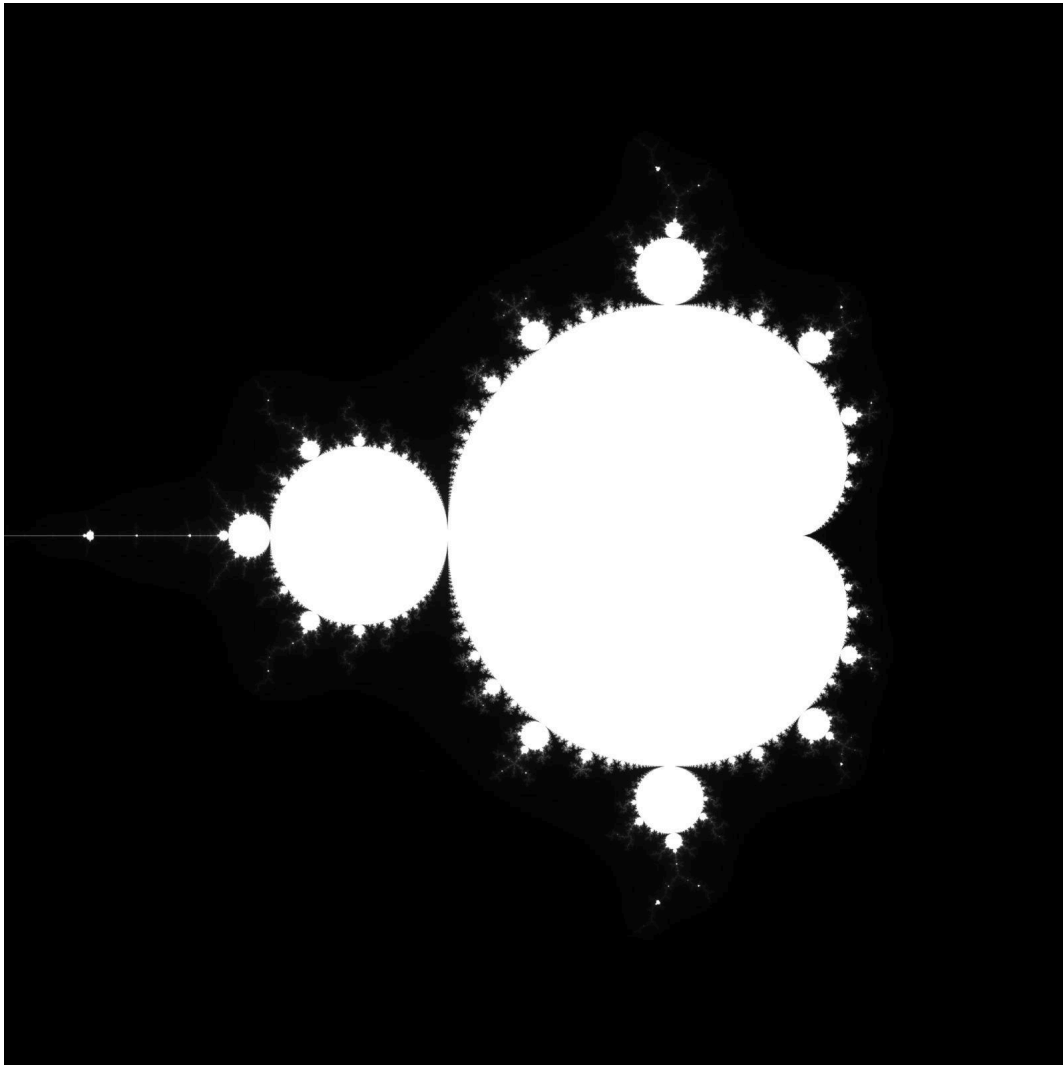


fig.26 resultado de la ejecución del código distribuido.

Evidencia de la Metodología Scrum.

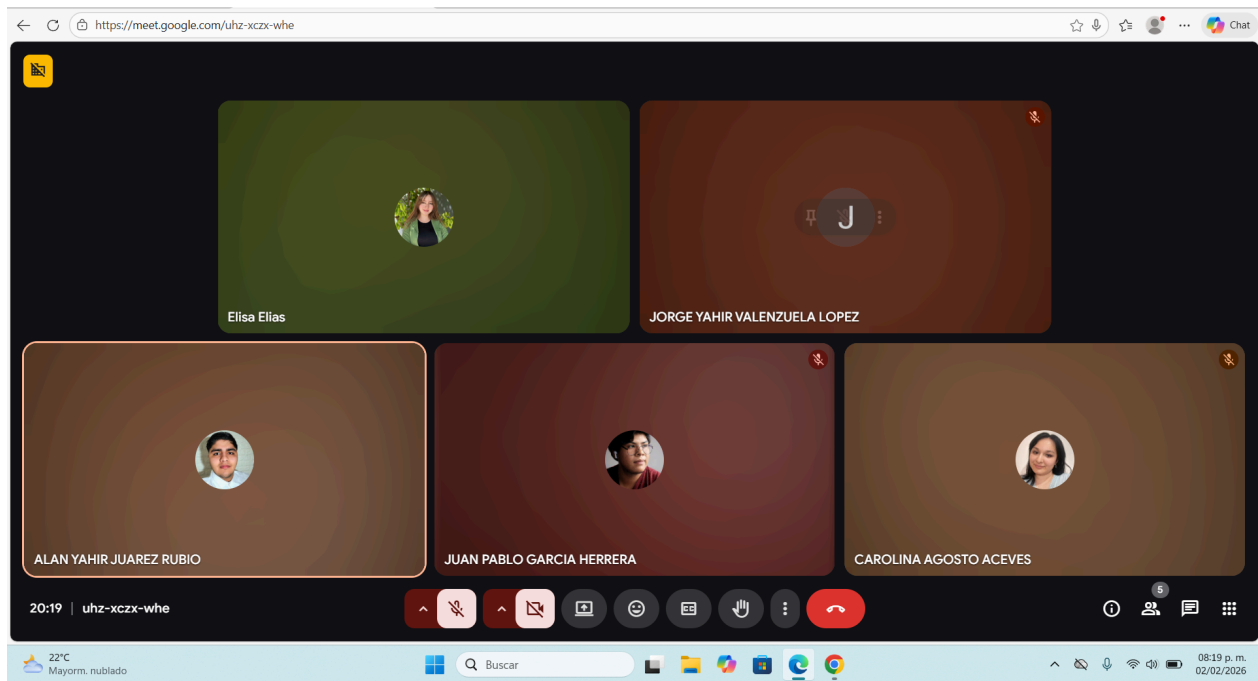


Fig. 23. Sesión en Meet para la organización del proyecto.

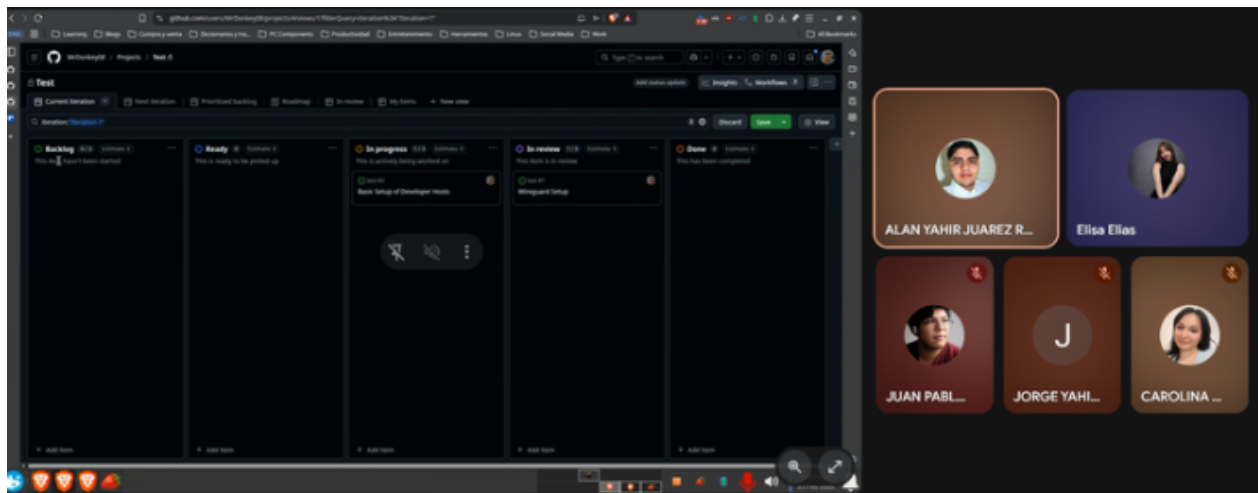


fig.23 planificacion del scrum

The screenshot shows a GitHub project board for a repository named 'Sistema Distribuido'. The board is filtered to show items with the status 'In review'. There are 6 items listed, each with a title, assignees, linked pull requests, and sub-issues progress. The items are numbered 1 through 6.

Title	Assignees	Linked pull requests	Sub-issues progress	Review
1 Configuración Básica de los Hosts de los Desarrolladores #2	AEYahir, Elisa Elias0...			
2 Documentación acerca del Conjunto de Mandelbrot #5	AEYahir			
3 Diagrama de la Red y los Roles de los Hosts y sus Nodos #6	MrDonkey08			
4 README.md #7	Juamp843			
5 Pruebas de rendimiento con iperf3 #8	AEYahir, Elisa Elias0...			
6 Presentacion #10	Juamp843 and Nati...			

fig.24 actividades de scrum desde github la mayoría en review

Principales retos técnicos y aprendizajes.

Reto 1 — VPN Completa y Verificada con WireGuard

Se logro implementar una topologia hub-and-spoke , generar pares de llaves para cada nodo ,configurar wg0.conf y que todos los nodos pudieran conectarse con handshake reciente

Reto 2 - Red de Contenedores sobre la VPN

En este reto se implementó una red de contenedores que funciona sobre la VPN del clúster. Esto permite que los contenedores desplegados en diferentes nodos se comuniquen entre sí como si estuvieran en la misma red local.

Reto 3 - Algoritmo Distribuido en Rust

Implementar un binario Rust corriendo dentro de los contenedores que pueda enviar y recibir mensajes entre nodos.

Problemas Encontrados y Decisiones Técnicas Tomadas.

Configuración de la VPN en Wireguard del lado del Servidor

Del lado del servidor fue necesario configurar el router para permitir enrutar correctamente el tráfico de la VPN creada con Wireguard asegurando la comunicación entre los nodos del sistema distribuido.

Así mismo fue necesario configurar el ICS (Conexión Compartida a Internet) para permitir a los distintos nodos y comunicarse dentro de la red local.

Repositorio en Github.

Enlace al Repositorio

https://github.com/MrDonkey08/Quantum-Core_Sistema-Distribuido.git

Conclusión.

Durante el desarrollo de este proyecto se adquirieron conocimientos prácticos sobre sistemas distribuidos particularmente sobre la ejecución de tareas en paralelo, el manejo de contenedores y la orquestación de Kubernetes incluyendo la gestión de pods , despliegues y comunicación entre servicios.

Además se comprendió la importancia de la conectividad en entornos distribuidos mediante el uso de una red VPN , identificando retos como la comunicación entre nodos , configuración de red y resolución de problemas de acceso.

El proyecto permitió identificar y resolver problemas reales asociados a la implementación de sistemas distribuidos , conectividad y políticas de comunicación.

Referencias.

Ahmed, K. (2026, 26 febrero). Linux Roadmap. *roadmap.sh*. <https://roadmap.sh/linux>

Basic writing and formatting syntax - GitHub Docs. (s. f.). GitHub Docs.

<https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Checking your browser - reCAPTCHA. (s. f.).

<https://www.kaggle.com/code/faraahanwaaar/car-sales-price-prediction/input?authuser=0>

Getting started. (s. f.). <https://rust-lang.org/learn/get-started/>

Hira, Z. (2025, 12 febrero). *Learn Linux for Beginners: From Basics to Advanced Techniques*

[Full Book]. freeCodeCamp.org.

<https://www.freecodecamp.org/news/learn-linux-for-beginners-book-basic-to-advanced>

JimcostDev. (s. f.-a). *GitHub - JimcostDev/my-docker-notes: Referencia rápida y completa para comandos Docker*. GitHub. <https://github.com/JimcostDev/my-docker-notes>

JimcostDev. (s. f.-b). *GitHub - JimcostDev/my-docker-notes: Referencia rápida y completa para comandos Docker*. GitHub. <https://github.com/JimcostDev/my-docker-notes>

Learn Kubernetes basics. (s. f.). Kubernetes.

<https://kubernetes.io/docs/tutorials/kubernetes-basics/>

Shotts, W., Jr. (s. f.). *LinuxCommand.org: Learn the Linux command line. Write Shell scripts*.

Copyright 2000-2026, William Shotts, Jr. <https://linuxcommand.org/>